

Network Traffic Identification with Convolutional Neural Networks

Akshit Jain
avj2668@rit.edu

April 2, 2018

Abstract

Network traffic identification plays a major role in modern-day network monitoring systems. Most systems identify network traffic based on features such as, flow statistics, static signatures and port numbers. Identifying network traffic is essential, because plenty of information regarding a network flow can be learned by knowing the application protocol associated with it. However, the challenge for traffic classification is to identify features in the flow data. This paper explores the issue of network traffic identification with neural network and deep learning. A convolutional neural network (CNN) with different optimization algorithms is trained to predict application protocols based on flow data. The image processing metaphor of the CNN model is extended to naturally fit to the curated dataset. Protocol labels with a high frequency distribution are easily detected by the model, and the results show that the CNN model works equally well on network flow data. A discussion on the performance of different optimization algorithms used with the CNN model is presented.

1 Introduction

The amount of network traffic being generated by Internet of Things (IoT) is increasing exponentially and it will soon start asking questions about the capabilities of the existing network monitoring and management systems. The network traffic comprises of various types of protocols and identifying each protocol is important for Quality of Service (QoS) as well as effective network management of resources in IoT networks and policies, as the network is required to respond based on the traffic protocol information.

There are several approaches to identify network traffic: payload-based, port-based and flow statistics-based. These approaches are often inaccurate and rely heavily on the understandings of the specialists. The need of the hour is to have a network identification system that does not require human intervention, but yet provides high accuracy.

The payload-based approach requires Deep Packet Inspection (DPI) of the flow data. This method looks for known patterns (i.e. signatures) inside the packets, and provides with the finest detection rate out of all the approaches mentioned above. However, there are some costs associated with this approach: real-time handling of large amounts of traffic on high-speed network is computationally expensive [1], and an up-to-date database of signature patterns needs to be maintained, in order to protect against all foreign attacks.

This paper defines a new payload-based method to identify the protocol/service being used on the network. The paper is organized as follows: Section 2 reviews the traditional methods and related work for network traffic identification. Section 3 details the work performed, i.e. data pipeline, selected dataset and model description. Section 4 describes the results obtained and provides a through analysis, and finally section 5 provides discussion and conclusion.

2 Literature Review

Researchers have explored different approaches to resolve the problem of network traffic identification. Identifying traffic based on predefined ports (for example, the default port for SSL is 443 and the standard HTTP port is 80) is one of the oldest methods, however, this method is not reliable as newer protocols do not use registered ports or ports used by other services.

Payload-based traffic identification is an alternative approach that involves the exhaustive search of reliable signatures but with more promising accuracy [2]. For every protocol, a signature is a fixed and unique sequence of strings or hex values found in the payload data [2]. A payload-based approach is simple and since it provides better accuracy, most network monitoring and management systems adopt it. However, the signatures need to be updated every time new protocols are developed or whenever there is a change in the specification of a protocol. This is very tedious and requires developers to manually update the signatures every time. These shortcomings make the classic approach of payload-based detection susceptible, as malware developers can regularly change their code or the way it is packaged to make sure it does not produce the same signature as previous versions, detection of which may have been added to existing signature lists of known bad code.

Comparing research results for the problem of network traffic identification using neural networks is difficult because the dataset used to train/test the model, and the performance metrics and optimization algorithms being applied to the model are very different. There are only a few related works that use neural networks to address the problem of network traffic identification by studying the flow payload data. Nevertheless, researchers have used features found in packet headers to train traditional machine learning algorithms and neural networks to solve the traffic identification problem.

In [3] information from the headers of packets exchanged during the flow lifetime is extracted to build a time-series of feature vectors in order to train a combination of the RNN and CNN models to identify network traffic. The features employed in the learning process play an important role, and the model attains an accuracy of 0.9632, an F1 score of 0.9574, a precision of 0.9543 and a recall of 0.9632.

The authors in [4] apply various machine learning techniques (Support Vector Machine, Naive Bayes, C4.5) to address network traffic identification. The average accuracy reported was less than 80% using 23 features and the approach was only good enough to detect five services.

In comparison to signature-based identification, deep learning methods can spot malicious traffic that modifies to change its signature, as identification is based on the entire flow payload data, not just the static code. Therefore, this paper tries to address the issue of network traffic identification using flow payload data as features to train a Convolutional Neural Network (CNN).

3 Work Description

The following sections will provide detailed information about the data pipeline, selected dataset and the model architecture used to train a Convolutional Neural Network.

3.1 Data Pipeline

Figure 1 provides an overview of the data pipeline used to identify network application protocol using a CNN model. There are multiple phases through which the payload data needs to pass through before it can be used to train a deep learning model. In the training stage, after the dataset is constructed, data is loaded, transformed (features: reshaped to spatial dimension, labels: one-hot-encoded) and finally fit to the model. The test data is used to evaluate the performance of the deep learning model that was earlier trained on the

training dataset. Finally, the labels (ground-truth protocols) associated with the test data can be compared to the predictions made by the CNN model.

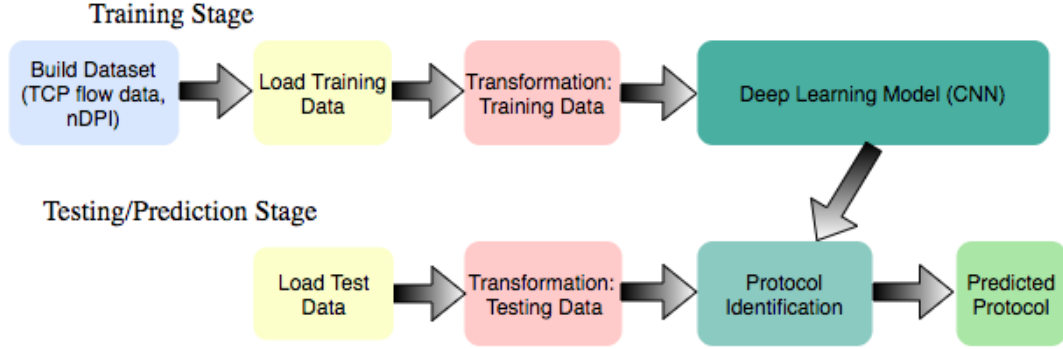


Figure 1: Data pipeline for Network Traffic Identification

3.2 Selected Dataset

For this study, network traffic was collected during the national CPTC held at RIT in November, 2017. From the collected traffic, 34,929 TCP flows were extracted. These flows contained 24 unique protocol labels, with a fairly unbalanced frequency distribution. Figure 2 shows the protocol names and the frequency distribution for the most frequent protocols.

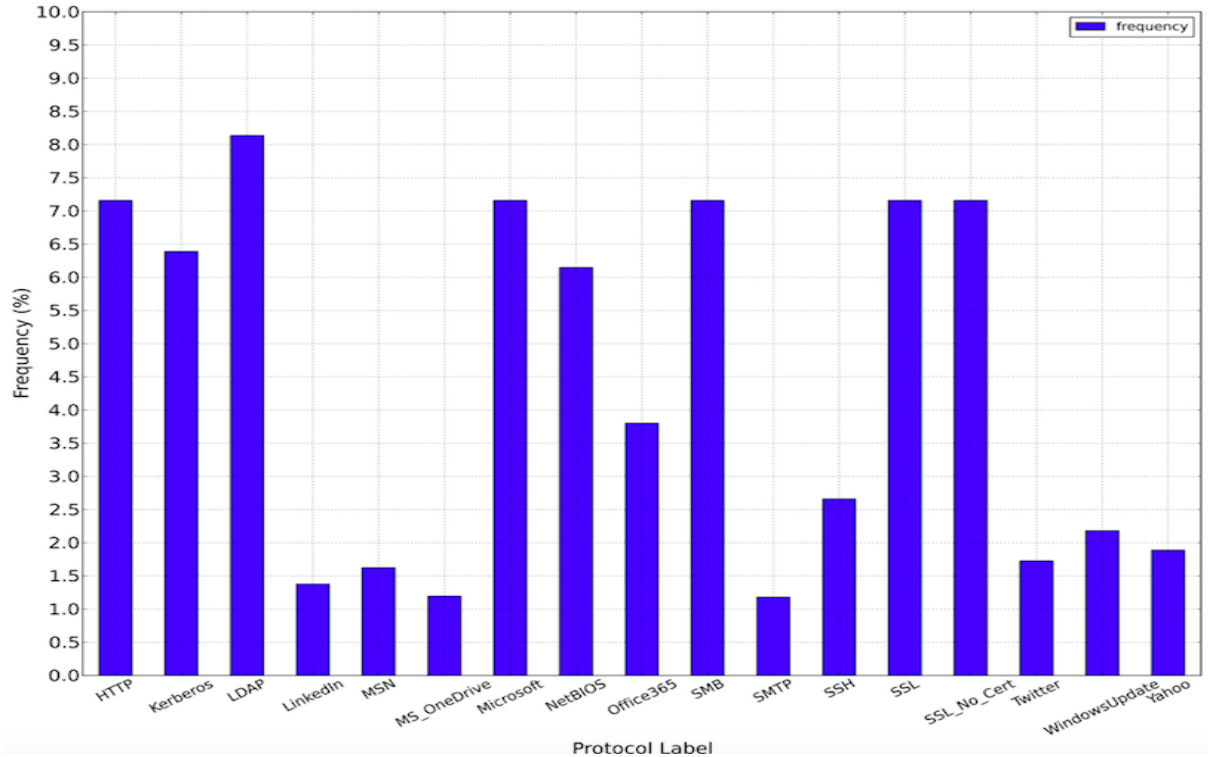


Figure 2: Frequency distribution of application protocol labels

Every TCP flow is identified with a specific protocol. In order to train and evaluate the deep learning model (CNN), each flow needs to be assigned a ground-truth protocol. The protocol is not available in the packet headers, and can only be found in the payload data. nDPI tool [5] is applied to the TCP flows in order to extract the protocols associated with the respective flows. nDPI applies a Deep Packet Inspection (DPI) technique to detect application protocols. Since, DPI (payload-based approach) has the finest detection rate, it is safe to assume the output of a DPI tool to be our best guess to the ground-truth protocol. For this study, only TCP flows were considered and the flows which nDPI did not detect were not considered.

Only the first 1024 bytes of a flow were extracted using the tcpflow program [6], and flows with less than 1024 bytes were discarded. This was done in order to keep the training consistent. Next, the hexadecimal output of the tcpflow program was mapped to its respective integer values.

TCP flow data is represented by an integer from 0 to 255 and is very similar to the examples of Pattern Recognition. A payload can be thought of as a document or a picture, and each byte as a word or a pixel. Since research [7] suggests that features of images and documents can be learnt by various deep learning models (e.g. CNN and RNN), it is realistic to expect the models to perform equally well, if not better, in the field of information security.

Finally, the dataset is built using the extracted TCP flows. Hence, the dataset consists of 34,929 TCP flows along with their associated application protocol.

3.3 Model Description

A 1-dimensional Convolutional Neural Network with different optimization algorithms has been considered. The CNN model with SGD-Momentum optimizer has the best detection performance. This section describes the different layers used to build the CNN model.

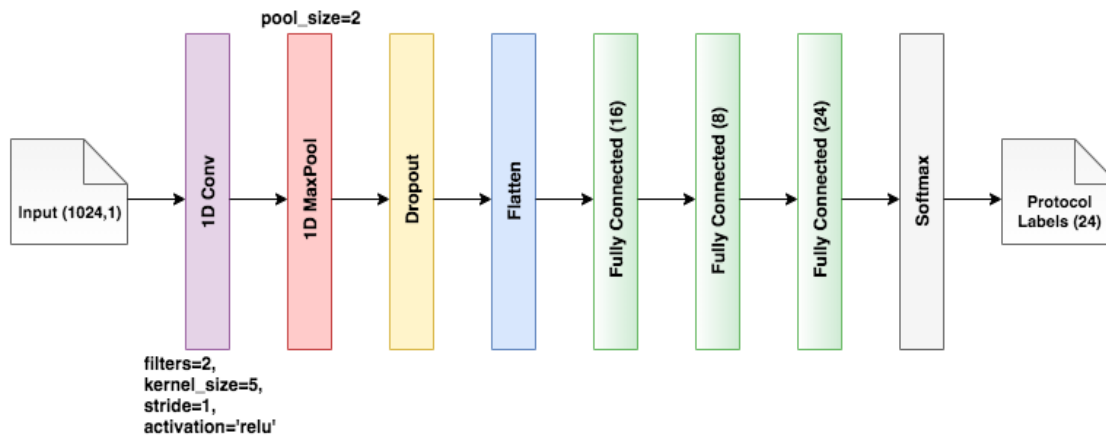


Figure 3: Architecture of a 1-dimensional CNN model

The architecture of a basic 1-dimensionnal CNN model is shown in Figure 3. CNN models have been the go-to model to perform image classification, where feature engineering is done automatically by the network. It is also successfully applied to natural language processing and recommender systems. The filter (kernel) in the convolutional layer automatically extracts location invariant patterns from input data without any human supervision. Complex features can be extracted by chaining multiple CNNs.

The image-processing or NLP metaphor can be used to apply the model to a very different dataset. In order to do that, payload bytes are treated as pixels/word embeddings. Image pixels and word embeddings [8], are both locally correlated; correspondingly, payload bytes in a network packet capture exhibit a similar correlated local behavior.

Whether the input is an image or a vector of words, all CNN models follow a similar structure. The number of different types of layers can vary depending on the problem being solved. In our case, we perform a single operation of convolution + pooling, followed by a dropout and a flatten layer, and finally a number of fully connected layers. Since, we are performing multiclass classification the output is softmax.

The main building block of CNN is the convolutional layer. Convolution is a mathematical operation to merge two sets of information [9], in other words, convolution filters need to convolve with the input and produce a feature map. The output of the convolution operation is directly dependent on the length of the filter and the stride length of the convolution.

For any kind of neural network to be powerful, it needs to contain non-linearity. This is achieved by passing the result of the convolution operation through a relu activation function.

After relu function is applied to the feature maps from the convolutional layer, max pooling operation is performed on them to reduce the dimensionality. Max pooling takes the size of the window and stride as parameters, and selects the maximum value in the pooling window. This operation reduces the number of parameters, which both combats overfitting and curtails the training time. The output of this operation is a downsampled feature map which still holds important information.

Dropout is the most popular regularization technique for deep neural networks. It provides a generalization of results for unseen data by dropping out (setting to zero) a percentage of outputs from the previous layer. This forces the network to not over-rely on any specific input, avoiding overfitting and improving generalization.

After convolution, max pooling and dropout layers; a couple of fully connected layers are added to wrap up the CNN architecture. The fully connected layer expects a 1-dimensional vector of numbers, therefore, a flatten layer is added after the dropout layer. The output of the flatten layer becomes the input to the fully connected layer. The final output is a result of the softmax activation function which scales the values of the output nodes such that they represent probabilities and sum up to 1.

Layer (type)	Input Shape	Output Shape
input_1 (InputLayer)	(None, 1024)	(None, 1024)
conv1d_1 (Conv1D)	(None, 1024)	(None, 1020, 2)
max_pooling1d_1 (MaxPooling1)	(None, 1020, 2)	(None, 510, 2)
dropout_1 (Dropout)	(None, 510, 2)	(None, 510, 2)
flatten_1 (Flatten)	(None, 510, 2)	(None, 1020)
dense_1 (Dense)	(None, 1020)	(None, 16)
dense_2 (Dense)	(None, 16)	(None, 8)
dense_3 (Dense)	(None, 8)	(None, 24)

Table 1: Summary of the CNN model with input and output shape at each layer

4 Results

This section analyzes the results obtained when applying the CNN model to identify network traffic. The influence different optimization algorithms have on the training stage is analyzed using various plots (loss vs. epoch and accuracy vs. epoch) and metrics.

In order to better understand the identification quality of the model, and considering the unbalanced distribution of the protocol labels, the following performance metrics are used: accuracy, precision, recall, and F1. Out of all the metrics considered, F1 is the most significant metric for this problem. The F1 score provides a better indication of the quality of protocol identification for an unbalanced dataset, and can be interpreted as a weighted average of the precision and recall, where an F1 score reaches its best value at 1 and worst score at 0.

Keras with Tensorflow backend is used to implement the model, and the performance metrics are calculated using the python package scikit-learn [10].

For this study, different optimization algorithms were used to check their suitability with the CNN model, and subsequently address the issue of network traffic identification. The following optimizers were experimented with: Adam, RMSprop, Stochastic Gradient Descent (SGD), and its improved versions SGD-Momentum and SGD-Nesterov. All the optimizers were tested under similar conditions, including the number of iterations (i.e. 200 epochs).

The dataset curated for this study has 24 distinct protocol labels to be detected. Identifying network traffic is a multi-class classification problem, and there are two possible ways to present results for such problems: aggregated and One-vs-Rest results.

For One-vs-Rest, the focus is on a specific class (protocol label) and the other classes are considered as single alternative class. This helps to simplify the problem to a binary classification task for each individual class. As for aggregated results, the results are summarized for all the classes.

There are multiple ways to perform the aggregation (samples, macro, micro, weighted), differing in the way the averaging process is done.

The performance metrics in Table II are aggregated metrics using a weighted average. The aggregated F1, precision and recall metrics are calculated using the weighted average provided by scikit-learn.

The metric values are presented with a color code to make it easier to rank the results for the different optimizers. In this color code, dark green color denotes better results whereas red color denotes worse results. From Table II, it is evident that the CNN model with optimizer SGD-Momentum gives the best results for both accuracy and F1.

Optimizer	Accuracy	F1	Precision	Recall
SGD-Momentum	0.7781	0.7790	0.7949	0.7781
Adam	0.7651	0.7690	0.7959	0.7651
SGD-Nesterov	0.7591	0.7575	0.7729	0.7591
RMSprop	0.7083	0.7097	0.7521	0.7084
SGD	0.6588	0.6432	0.6916	0.6588

Table 2: Classification performance metrics (aggregated) vs optimizers employed (model CNN)

Figure 4 provides the One-vs-Rest metrics for the classification of the most frequent protocol labels (results obtained using SGD-Momentum with CNN model). An important observation in Figure 4 is that for all the protocol labels with a frequency higher than 6% (Figure 1), excluding SSL protocol, the model achieves an F1 score higher than 0.85.

Protocols such as SSH and SSL contain encrypted traffic which makes classification difficult for any machine learning model, i.e. if the model is ignoring the entropy in the payload data. Entropy-based classification is a method that can be used to distinguish encrypted parts from plain-texts [11]. The model developed in this study does not take the entropy of encrypted packets into consideration.

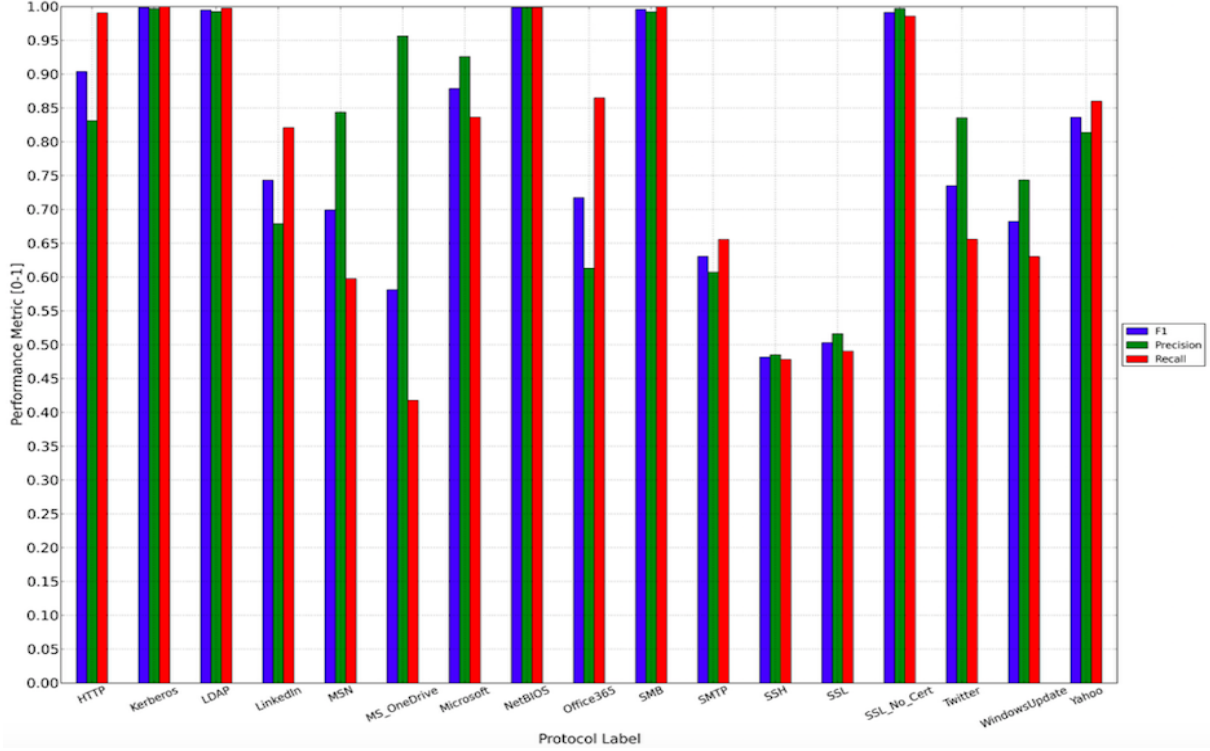


Figure 4: Performance metrics (one vs. rest) for the classification of protocol labels (more frequent)

4.1 Discussion: Performance of Optimization Algorithms

Optimization Algorithms influence the time it takes to train a deep learning model. There are other factors that determine the quality of the model (i.e. the performance metrics discussed earlier). However, these algorithms are executed iteratively by comparing various solutions till an optimum or a satisfactory solution is found (i.e. model convergence is achieved) in finite time.

The performance plots (See Appendix B) generated while training the model with different optimizers, suggest that the erratic RMSprop (Figure B4) is not a suitable algorithm for the CNN model to identify network traffic. However, there is very little to choose from the remaining four algorithms (i.e. Adam, SGD and its variants) in terms of performance.

Stochastic Gradient Descent (SGD) [12] is a popular stochastic optimization method. However, it takes time to converge (Figure B5) as it tends to oscillate in the wrong directions.

Momentum (Figure B1) helps the classic SGD to navigate along the relevant directions and softens the oscillations in the irrelevant directions. It remembers the direction of its previous step and simply adds a fraction of the direction to the current step. This increases the speed in the correct direction and helps address the issue of the classic SGD. However, there is one problem with momentum: when it gets very close to the goal, and since the momentum is very high, it does not know that it should slow down. This causes it to miss or oscillate around the minima.

In theory, SGD-Nesterov overcomes the problem of Momentum by starting to slow down early. In Momentum, the gradient is calculated and then a jump is made in that direction amplified by the momentum from the previous step. Nesterov does the exact same thing but in a different order: it makes a jump based on the stored information, then calculates the gradient and makes a minor correction. However, in this study, Nesterov (Figure B3) under-performs in comparison to SGD-Momentum. It also starts overfitting to the data under 50 epochs.

Adam performs (Figure B2) at par with SGD-Momentum. Loss value implies how well or

poorly a model behaves after each iteration of optimization. Reduction of loss after each, or several iteration(s) is ideal. Adam, along with Momentum consistently reduce their loss value through backpropagation. Even though there is a difference in validation loss and training loss, the difference is not very significant, it is also because training loss is calculated as a moving average over 1 epoch, whereas the validation loss is calculated after the learning phase of the same epoch.

5 Conclusion

In this study, we discussed how network traffic identification methods in current network monitoring systems are inaccurate and rely heavily on the understandings of the specialists.

Previous research works have applied deep learning and classical machine learning algorithms to high-level-header-based data from the network packets. This work describes a payload-based method to identify network traffic using neural network (CNN) and deep learning.

The results demonstrate that a CNN can be successfully applied to identify network traffic, giving an easy way to extend the image and text processing hypothesis to payload data. Our approach solves the problem of non-automation and poor adaptation in the traditional methods.

The gradient descent algorithms SGD-Momentum and Adam, work really well with the CNN model. Better F1 detection scores are reported with these two optimizers. The One-vs-Rest metrics for the classification of the most frequent protocol labels (using SGD-Momentum), reports excellent F1 scores for labels with a higher frequency distribution. Thus, having more data samples for every application protocol would have boosted the performance of the model many folds.

The model developed in this study does not take the entropy of encrypted packets into consideration. Hence, the model does not perform well for encryption protocols. Entropy-based classification is a method that can be used to distinguish encrypted parts from plaintexts [13].

For future work, we would like to analyze encrypted traffic and also experiment with different deep learning models, for example LSTM.

Acknowledgment

This project is partially supported by the National Science Foundation under Grant No. 1433736. I thank my advisor Bo Yuan, Ph.D. for his guidance and sharing his knowledge with me during the course of this project.

References

- [1] S. H. Lee, J. S. Park, S. H. Yoon, and M. S. Kim, “High performance payload signature-based internet traffic classification system,” in *2015 17th Asia-Pacific Network Operations and Management Symposium (APNOMS)*, pp. 491–494, Aug 2015.
- [2] B. C. Park, Y. J. Won, M. S. Kim, and J. W. Hong, “Towards automated application signature generation for traffic identification,” in *NOMS 2008 - 2008 IEEE Network Operations and Management Symposium*, pp. 160–167, April 2008.
- [3] M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, and J. Lloret, “Network traffic classifier with convolutional and recurrent neural networks for internet of things,” *IEEE Access*, vol. 5, pp. 18042–18050, 2017.
- [4] M. Shafiq, X. Yu, A. A. Laghari, L. Yao, N. K. Karn, and F. Abdessamia, “Network traffic classification techniques and comparative analysis using machine learning algorithms,” in *2016 2nd IEEE International Conference on Computer and Communications (ICCC)*, pp. 2451–2455, Oct 2016.
- [5] L. Deri, M. Martinelli, T. Bujlow, and A. Cardigliano, “ndpi: Open-source high-speed deep packet inspection,” in *2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*, pp. 617–622, Aug 2014.
- [6] K. Rahnamai, K. Gorman, and A. Gray, “Model predictive neural control of tcp flow in aqm network,” in *NAFIPS 2006 - 2006 Annual Meeting of the North American Fuzzy Information Processing Society*, pp. 493–498, June 2006.
- [7] A. Vailaya, M. A. T. Figueiredo, A. K. Jain, and H.-J. Zhang, “Image classification for content-based indexing,” *IEEE Transactions on Image Processing*, vol. 10, pp. 117–130, Jan 2001.
- [8] S. Kottur, R. Vedantam, J. M. F. Moura, and D. Parikh, “Visualword2vec (vis-w2v): Learning visually grounded word embeddings using abstract scenes,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4985–4994, June 2016.
- [9] L. Gao, P. Y. Chen, and S. Yu, “Demonstration of convolution kernel operation on resistive cross-point array,” *IEEE Electron Device Letters*, vol. 37, pp. 870–873, July 2016.
- [10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.
- [11] R. Alshammari and A. N. Zincir-Heywood, “Machine learning based encrypted traffic classification: Identifying ssh and skype,” in *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications*, pp. 1–8, July 2009.
- [12] Z. You and B. Xu, “Improving training time of deep neural network with asynchronous averaged stochastic gradient descent,” in *The 9th International Symposium on Chinese Spoken Language Processing*, pp. 446–449, Sept 2014.

Appendix A

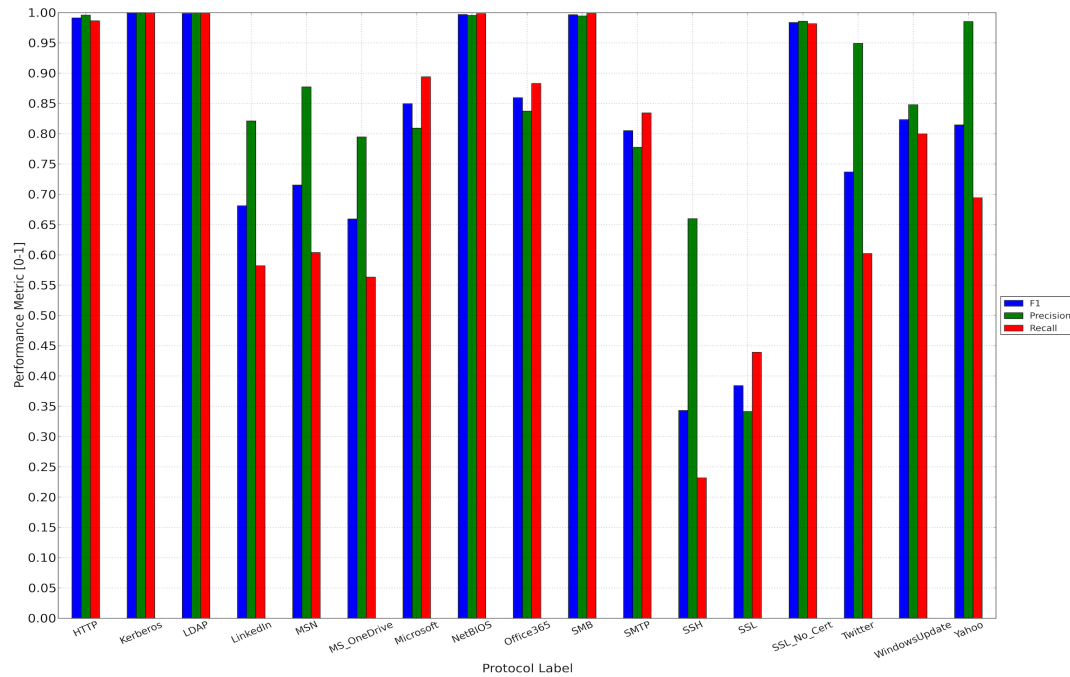


Figure A1: Performance metrics (one vs. rest) for protocol labels (more frequent) using Adam optimizer

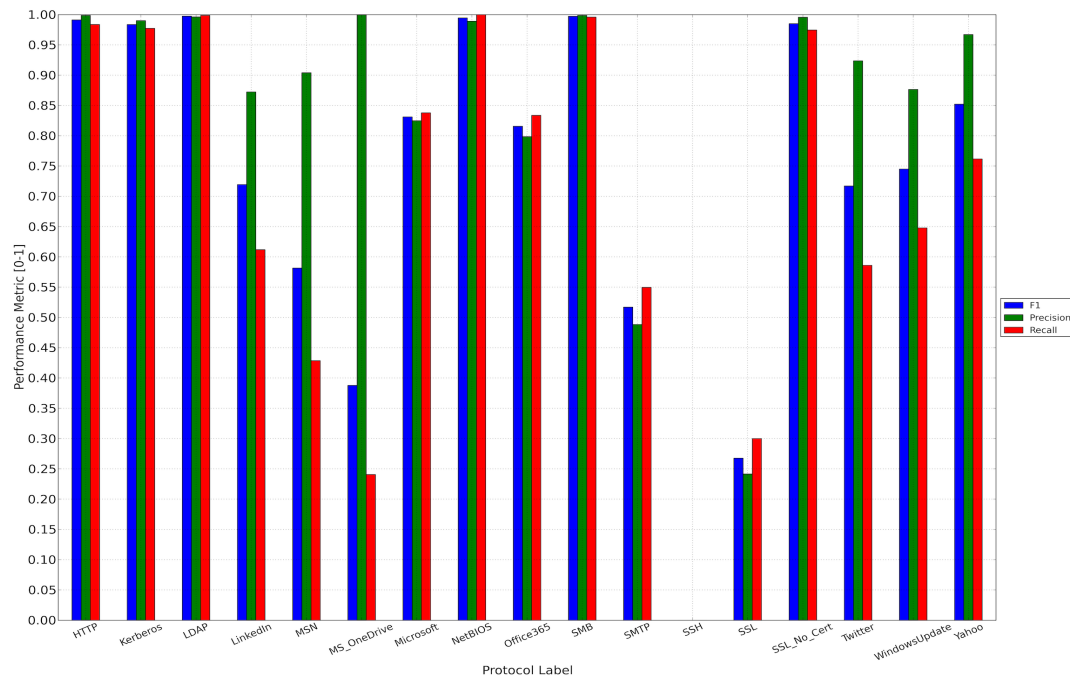


Figure A2: Performance metrics (one vs. rest) for protocol labels (more frequent) using RMSprop optimizer

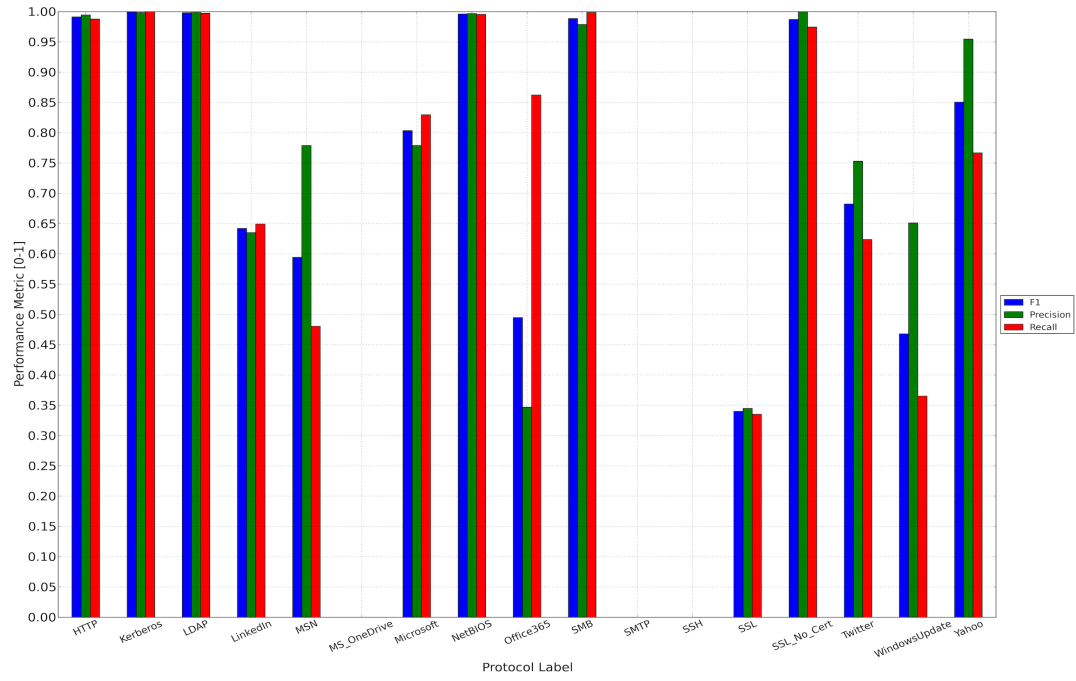


Figure A3: Performance metrics (one vs. rest) for protocol labels (more frequent) using SGD-Nesterov optimizer

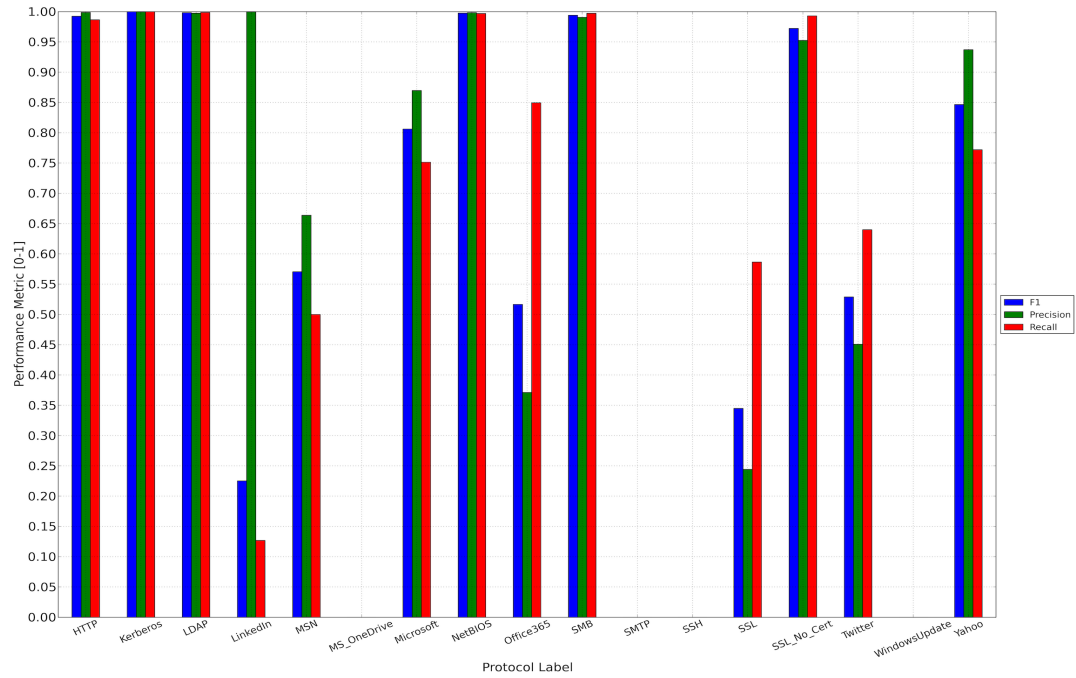


Figure A4: Performance metrics (one vs. rest) for protocol labels (more frequent) using SGD optimizer

Appendix B

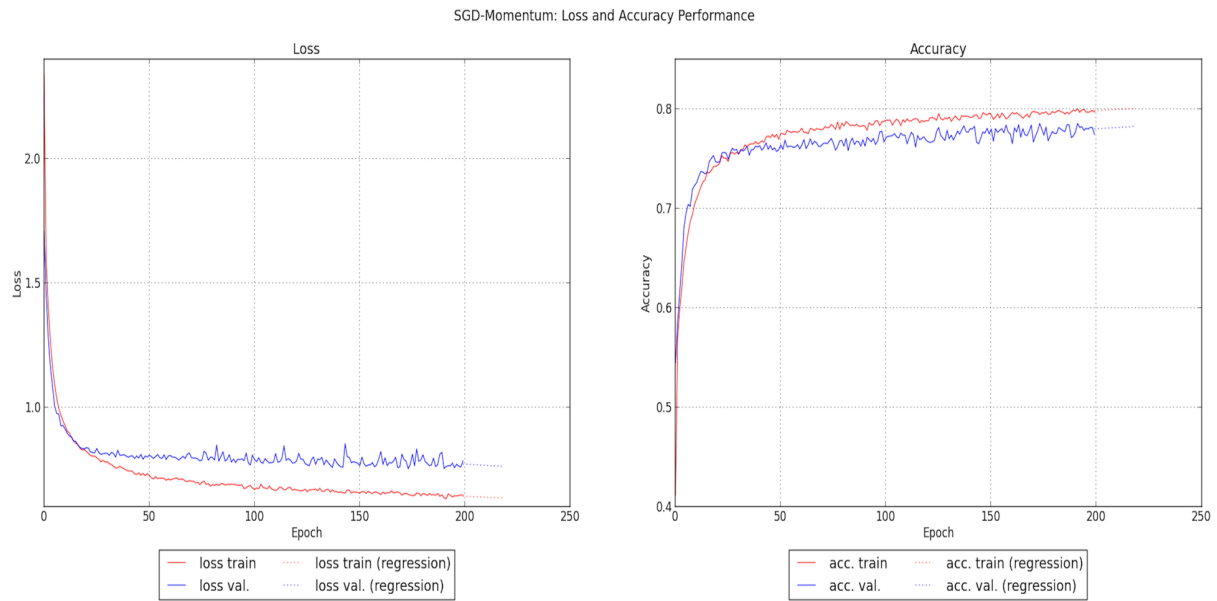


Figure B1: Loss vs. Accuracy Performance (SGD-Momentum optimizer)

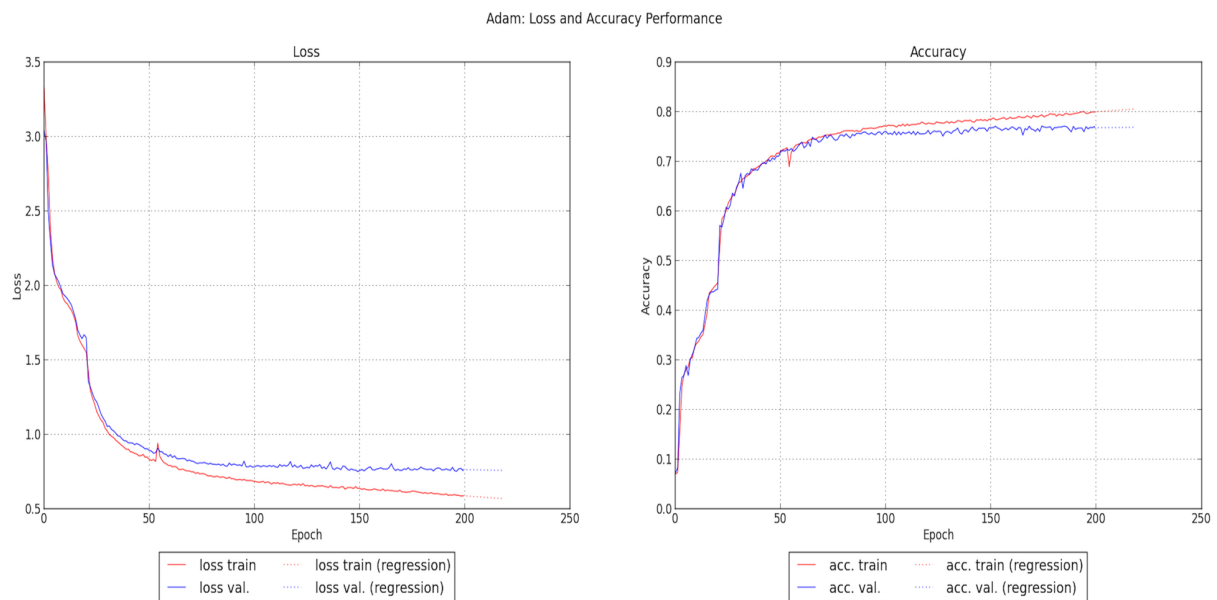


Figure B2: Loss vs. Accuracy Performance (Adam optimizer)

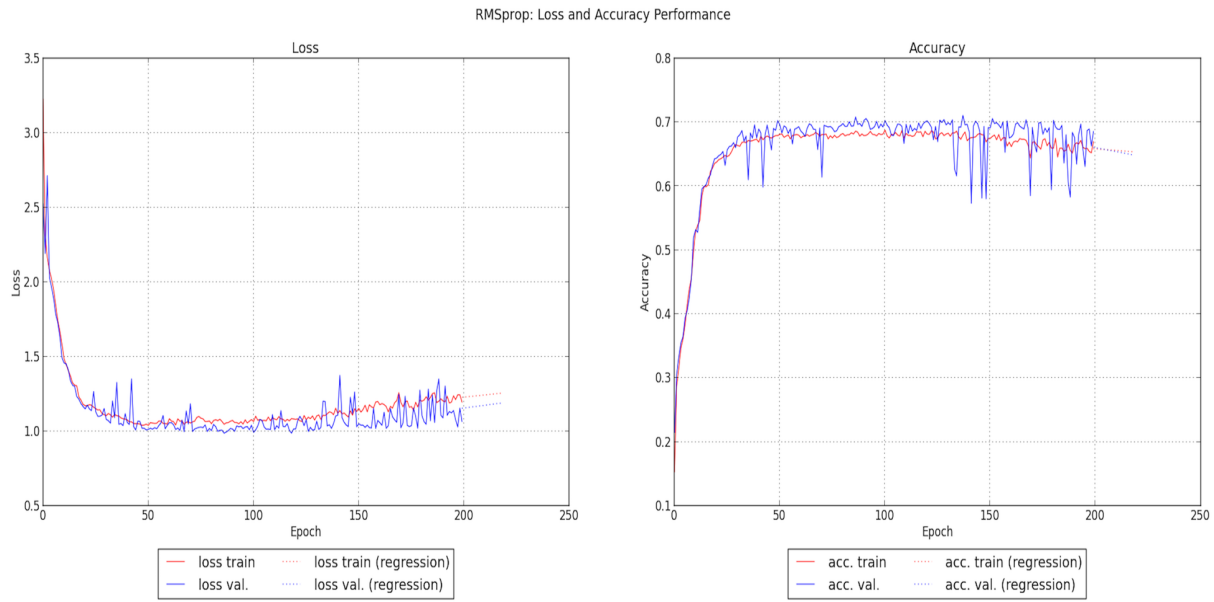


Figure B3: Loss vs. Accuracy Performance (RMSprop optimizer)

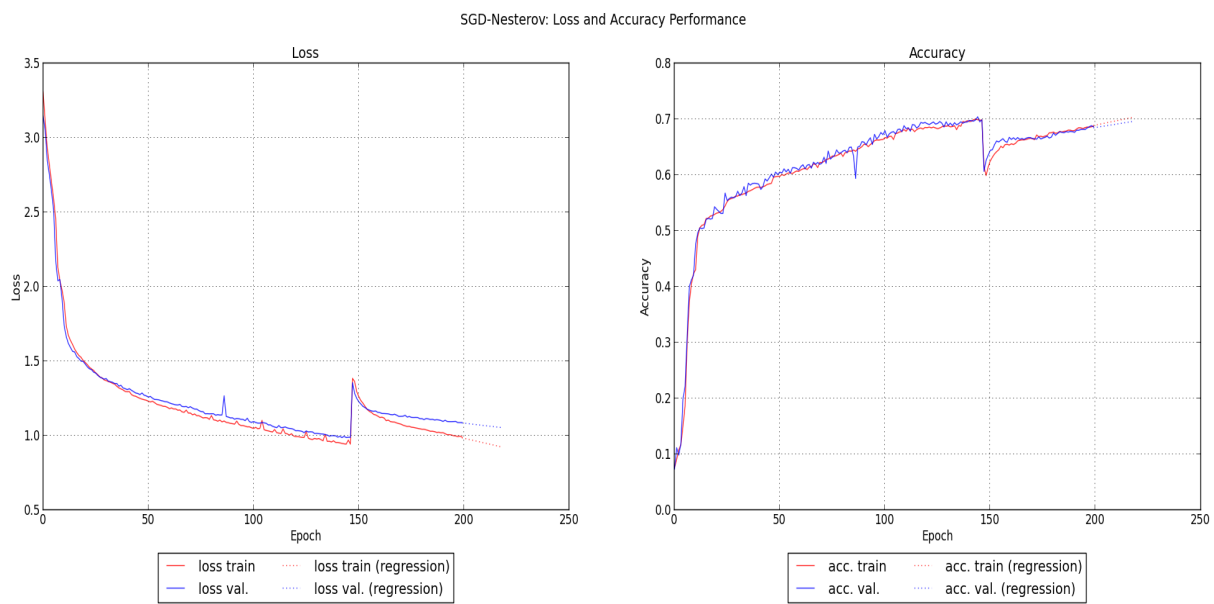


Figure B4: Loss vs. Accuracy Performance (SGD-Nesterov optimizer)

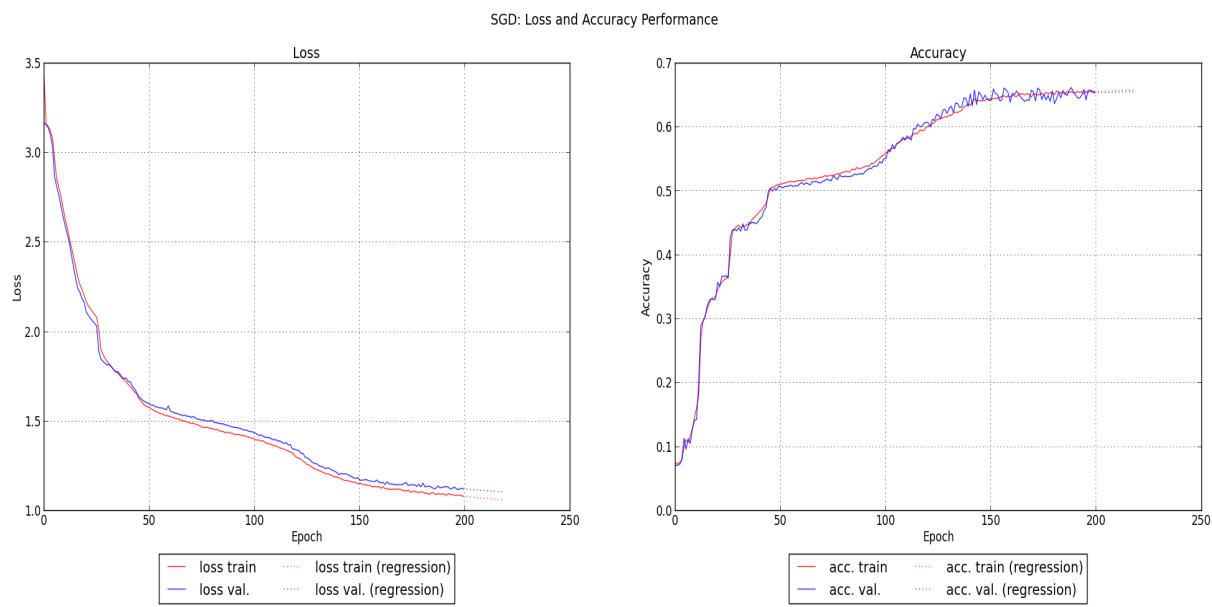


Figure B4: Loss vs. Accuracy Performance (SGD optimizer)